

Tavishi Session 22

Sunday, July 12, 2026 8:43 PM

Basics

- Variables ✓
- Data Types ✓
- Type Conversion ✓
- Input and Output ✓
- Comments ✓
- Operators ✓

Control Flow

- if ✓
- if-else ✓
- if-elif-else ✓
- Nested Conditions ✓
- Match Case (optional) ✓

Loops

- for Loop ✓
- while Loop ✓
- Nested Loops ✓
- Loop Control Statements (break, continue, pass) ✓

Strings

- String Indexing ✓
- String Slicing ✓
- String Methods ✓
- String Formatting ✓

Collections

- Lists ✓
- Tuples
- Sets
- Dictionaries

Collection Operations

- List Methods
- Tuple Operations
- Set Methods
- Dictionary Methods

Functions

- Function Definition
- Parameters and Arguments
- Return Statement
- Default Arguments
- Keyword Arguments
- Variable Length Arguments (*args, **kwargs)
- Scope (Local and Global)
- Lambda Functions
- map()
- filter()
- reduce()

String formatting :-

```
name = "Tavish"
print("Hello", name)
```

O/P — Hello Tavishi

```
name = "Tavishi"
print(name)
```

```
name="Tavishi"
print("Hello",name)
```

```
# String formatting
_name="Azaan"
age=24
profession="Software Engineer"
```

working="Remotely"

```
print(_name,"is a tutor at Codeyoung whose age is",age,"he is working",working,"as a", profession)
```

#String formatting using f string

```
print(f"{_name} is a tutor at Codeyoung whose age is {age} he is working {working} as a {profession}")
```

Data Types

Imagine a bank account.

A bank account contains different kinds of information:

- Account Holder Name → "Shanmuka"
- Account Balance → 50000.75
- Account Active → True
- Transaction IDs → [101, 102, 103] → Collection

All these values are different.

Python needs to know:

- Which value is text?
- Which value is a number?
- Which value is a yes/no answer?
- Which value is a collection?

This is where **Data Types** become important.

variables
↑
What kind of data it stores →

Variables:-

name = "Sophie" ←

age = 16 ←

wt = 45.75 ↘

Data Types

Single Value

These store one value at a time.

- Examples:
- int ✓ → age → 12
 - float ✓ → wt → 42.35
 - str ✓ → name → "Tejas"
 - bool ✓ → is active → false

age = 19 → unordered

that can change

Collection Data Types

These store multiple values together.

Examples:

- list
- tuple
- set
- dict

you can access by Index

	Data Type	Example	Mutable/Immutable	Ordered/Unordered	Real-Life Example
single	int	100	Immutable	N/A → Not Applicable	Roll Number
	float	99.5	Immutable	N/A	Product Price
	str	"Python"	Immutable	Ordered	Student Name
	bool	True	Immutable	N/A	Login Status
collection	list	[1,2,3]	Mutable	Ordered	Shopping Cart
	tuple	(1,2,3)	Immutable	Ordered	Coordinates
	set	{1,2,3}	Mutable	Unordered	Unique IDs
	dict	{'name':'Rahul'}	Mutable	Insertion-Ordered	Student Record

List ← collection data types

➤ List is a group of similar or different data types.

Ex- `marks = [14, 15, 8, 9, 11]`

`random_hobbies_age = ["volleyball", 24, True]`

Annotations for `random_hobbies_age`:

- `"volleyball"`: str
- `24`: int
- `True`: boolean
- g Know Python (points to the list)

nested list = `[[1, 2, 3]]`

loops
loop → inside → loop

A List allows us to store multiple values in a single variable.

Lists are:

- ✓ Ordered → we can access it using index
- ✓ Mutable → It can be change
- ✓ Allow Duplicate Values

```
subjects = ["Math", "Science", "English"]
```

```
# Print the entire list of subjects
```

```
print(subjects)
```

```
# Print the data type of the 'subjects' variable
```

```
print(type(subjects))
```

Accessing a list

```
#~ Accessing a List using index
```

```
subjects = ["Math", "Science", "English"]
```

```
#      0      1      2
```

```
print(subjects[0])
```

```
print(subjects[1])
```

```
print(subjects[2])
```

Modifying a List

```
subjects = ["Math", "Science", "English"]  
#         0      1      2
```

```
subjects[1] = "Physics"  
print(subjects)
```